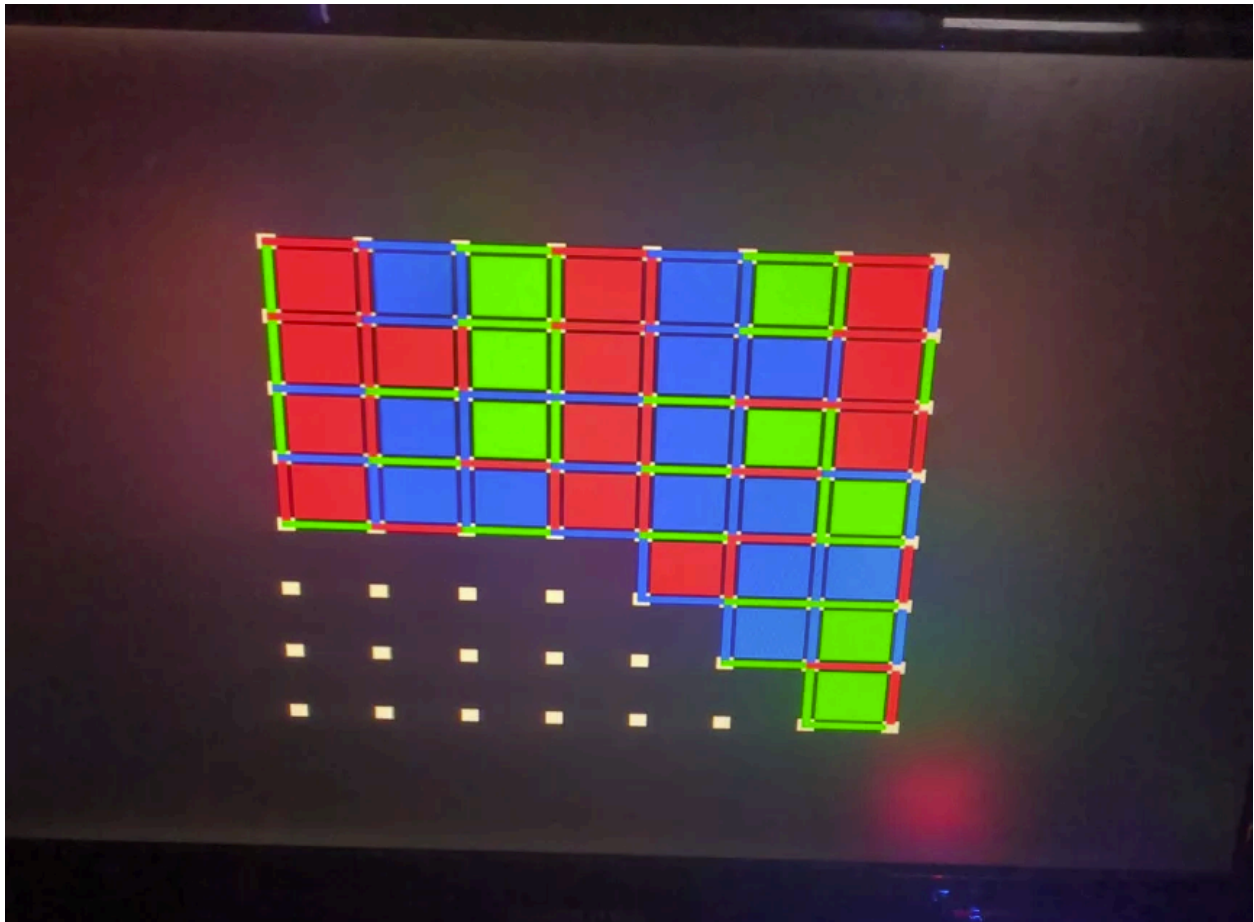


Dots & Boxes on VGA Monitor via FPGA



[Presentation and Demonstration video link](#)

Introduction

Dots and Boxes is a simple game that starts out with a square grid of dots, where players take turns to draw one line between two horizontal/vertical dots of their choice. Whichever player manages to draw a line that closes a square box, that box is marked as theirs and they get a point. This continues until the entire grid of dots is filled with boxes. Once the game is finished, the player with the most points (the most boxes finished) wins the game.

We will be using VHDL code on the Cyclone IV E FPGA and its native VGA port to a VGA monitor to display the game.

Software/Hardware Required

- Active-HDL
- Quartus II 13.1 (other versions may work)
- Cyclone IV E DE2-115 FPGA Board

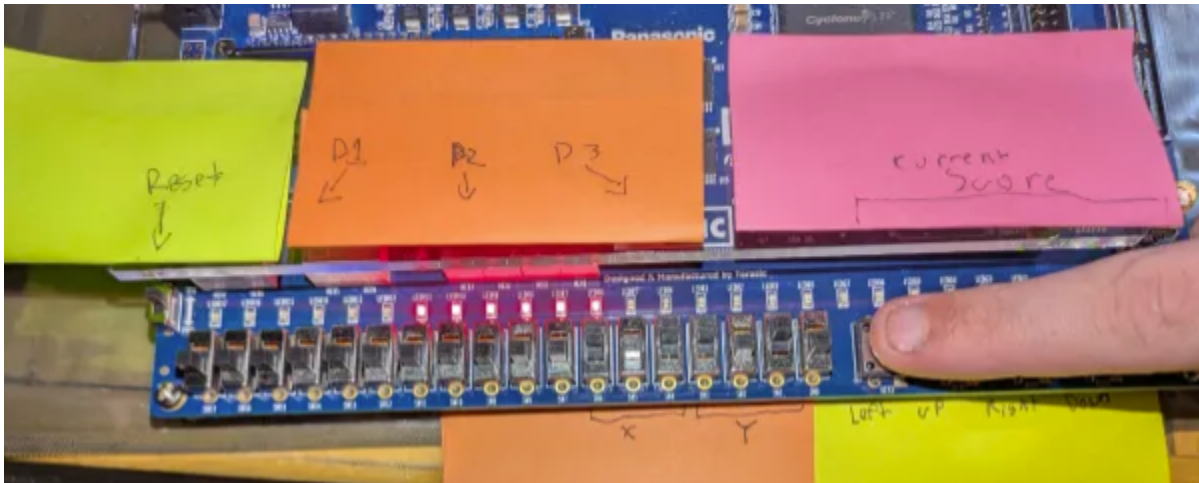
- Monitor with VGA 640x480 @ 60Hz capabilities
- USB Blaster installed

How to Play

This is a three player version of dots and boxes. The current player's score is displayed on the green LEDs in binary. The current player is displayed on the red LEDs by which 6 of them are lit up. Player 1: 6 leftmost leds are lit, player 2: 6 middle leds, player 3: 6 rightmost leds.

The 3 rightmost switches select the y coordinate in binary (0-7), and the 3 switches to the left of that similarly select the x coordinate. The 4 push buttons correspond to picking a direction left, up, right, or down. The leftmost switch is the reset toggle.

In the picture below, it is player 2's turn, with a score of 5, selecting the coordinate (3, 2), and direction left.



VGA Specifications

Clock

The VGA communication must send new data on a 25 MHz clock. Since the DE2 board has an internal clock of 50 MHz, we will divide it by two for the perfect clock signal.

Display Function Overview

In VGA, the column values are from 0 to 639, where other values are ignored. The row values are from 0 to 479, where other values are ignored. For a pixel to be colored on the screen, a combination of red, green, and blue (RGB) must be asserted on the rising edge of the clock. We use counters to keep track of the pixel coordinates.

There are signal delays and synchronization signals that have very specific timing which must be obeyed according to the figures below. For the simplicity of our calculations, those are moved to the end of the display interval in code, so the pixel drawing phase is correctly from values 0-479, 0-639.

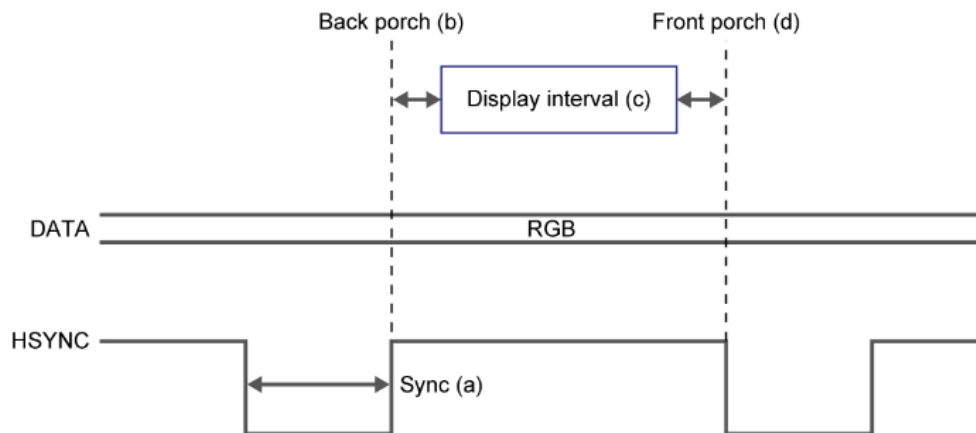


Fig. 1, VGA timing specification graph, horizontal case [Source](#) (Pg 54)

VGA mode		Horizontal Timing Spec				
Configuration	Resolution(HxV)	a(us)	b(us)	c(us)	d(us)	Pixel clock(MHz)
VGA(60Hz)	640x480	3.8	1.9	25.4	0.6	25

Fig. 2, Horizontal timing specification [Source](#) (Pg 54)

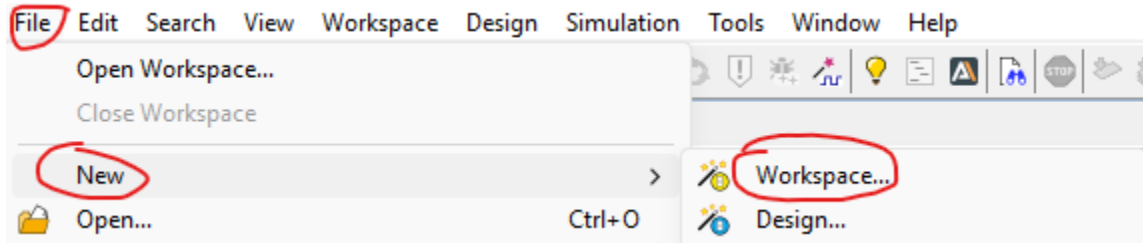
VGA mode		Vertical Timing Spec				
Configuration	Resolution(HxV)	a(lines)	b(lines)	c(lines)	d(lines)	Pixel clock(MHz)
VGA(60Hz)	640x480	2	33	480	10	25

Fig. 3, Horizontal timing specification [Source](#) (Pg 54)

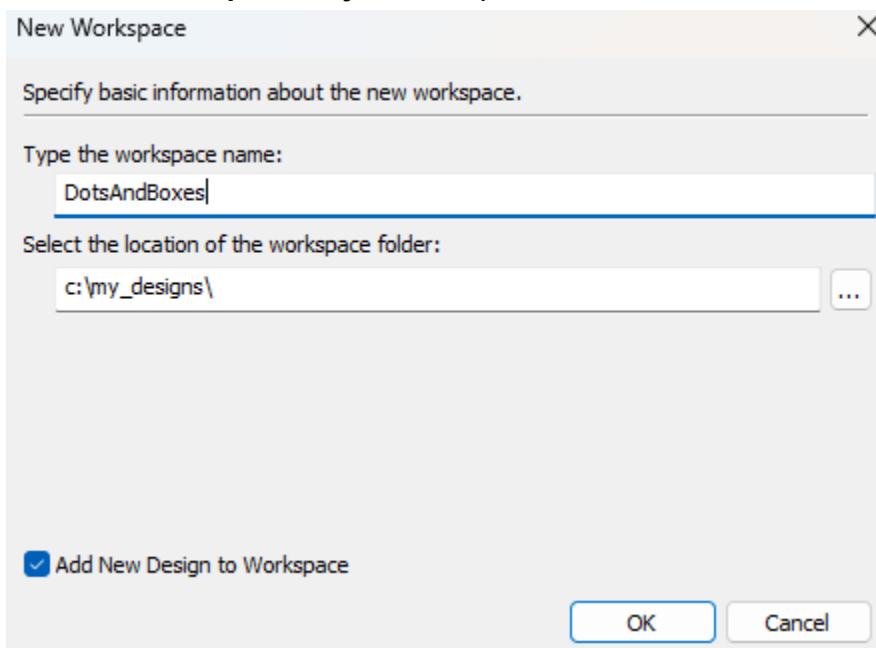
Compilation in Active-HDL

Creating the workspace

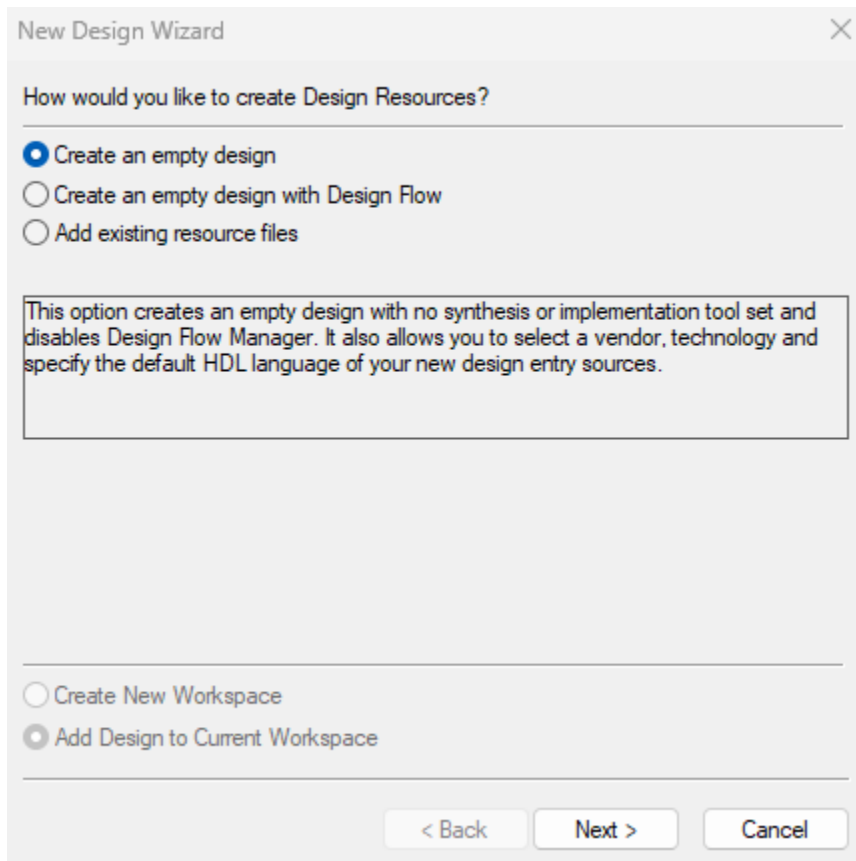
1. In Active-HDL, go to **File > New > Workspace**



2. Name the workspace as you wish (for this tutorial it will be named *DotsAndBoxes*)



3. Click on **Create an empty design**, then click **Next**



The image shows a 'New Design Wizard' dialog box with a close button (X) in the top right corner. The main question is 'How would you like to create Design Resources?'. There are three radio button options: 'Create an empty design' (selected), 'Create an empty design with Design Flow', and 'Add existing resource files'. Below these options is a text box containing the following text: 'This option creates an empty design with no synthesis or implementation tool set and disables Design Flow Manager. It also allows you to select a vendor, technology and specify the default HDL language of your new design entry sources.' At the bottom of the dialog, there are two more radio button options: 'Create New Workspace' and 'Add Design to Current Workspace' (selected). At the very bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'.

New Design Wizard

How would you like to create Design Resources?

☒ Create an empty design

☐ Create an empty design with Design Flow

☐ Add existing resource files

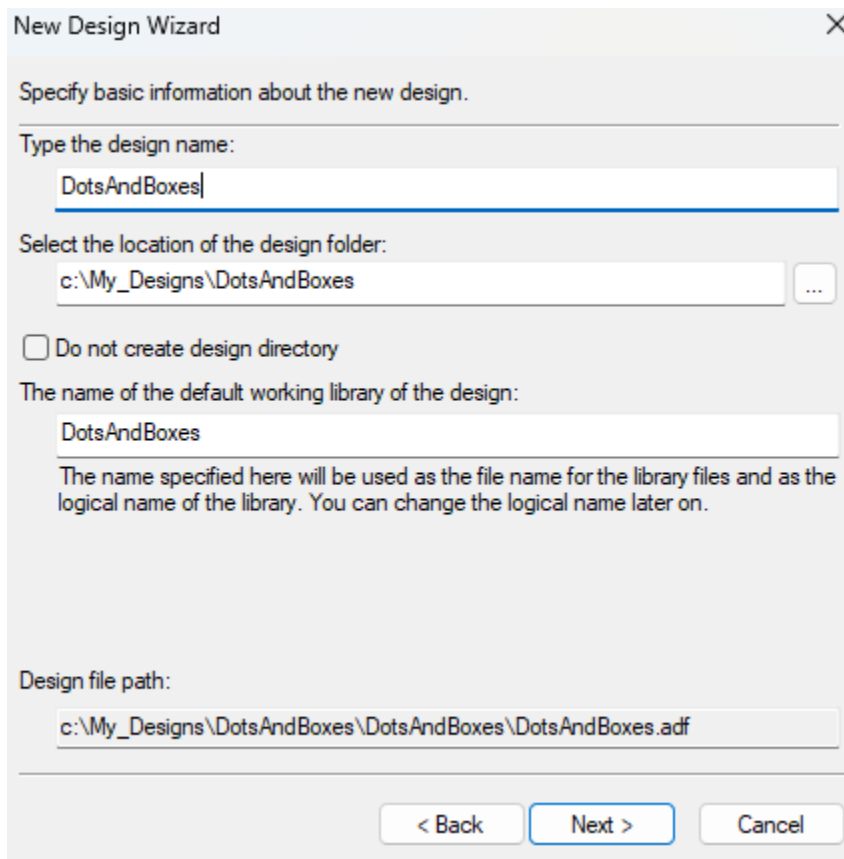
This option creates an empty design with no synthesis or implementation tool set and disables Design Flow Manager. It also allows you to select a vendor, technology and specify the default HDL language of your new design entry sources.

☐ Create New Workspace

☒ Add Design to Current Workspace

< Back Next > Cancel

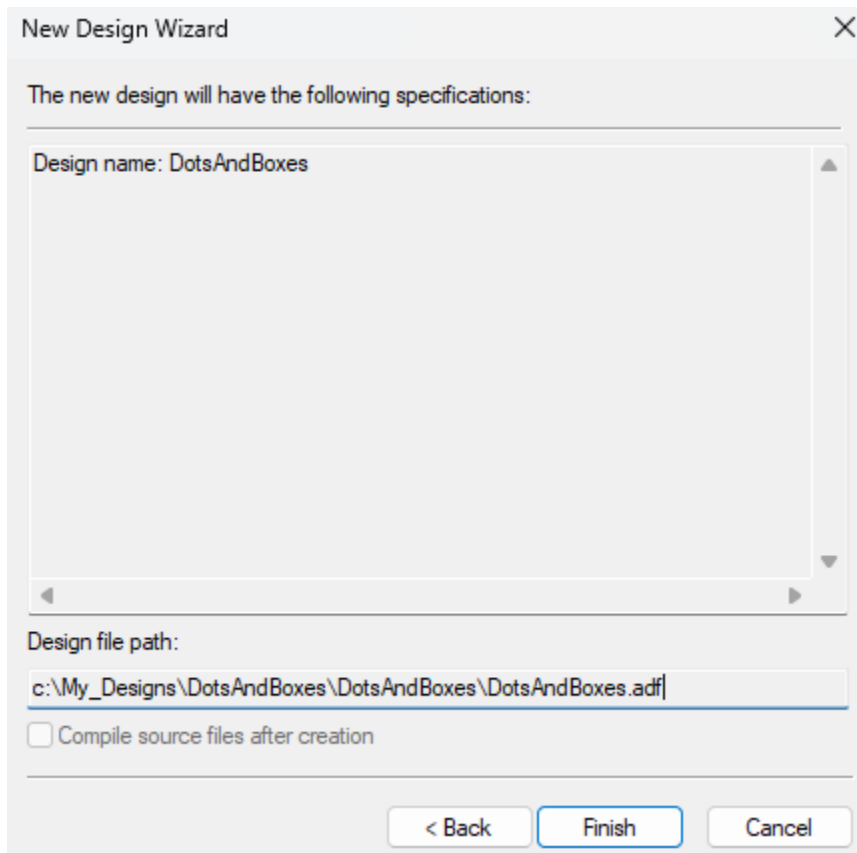
4. Leave following screens as default until you reach this screen, then name the design *DotsAndBoxes*, then click *Next*



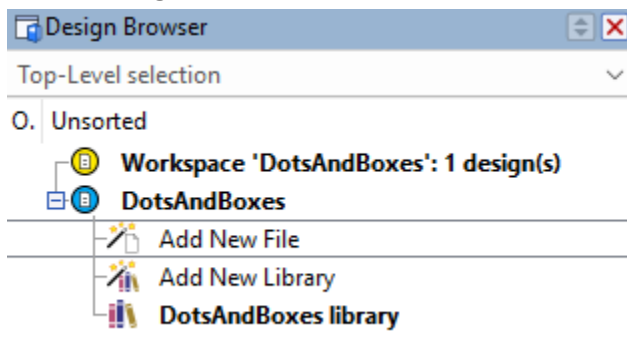
The image shows a 'New Design Wizard' dialog box with a close button (X) in the top right corner. The dialog is titled 'New Design Wizard' and contains the following fields and options:

- Specify basic information about the new design.**
- Type the design name:** A text box containing 'DotsAndBoxes'.
- Select the location of the design folder:** A text box containing 'c:\My_Designs\DotsAndBoxes' and a browse button (...).
- ☐ Do not create design directory
- The name of the default working library of the design:** A text box containing 'DotsAndBoxes'. Below this text box is a note: 'The name specified here will be used as the file name for the library files and as the logical name of the library. You can change the logical name later on.'
- Design file path:** A text box containing 'c:\My_Designs\DotsAndBoxes\DotsAndBoxes\DotsAndBoxes.adf'.
- At the bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'.

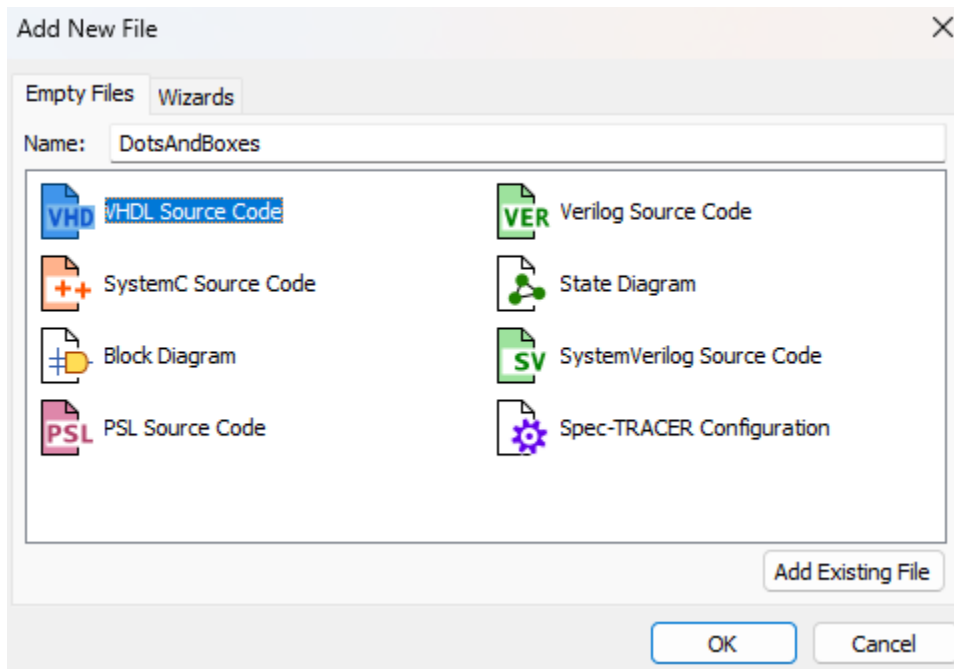
5. Once on this screen, click **Finish**



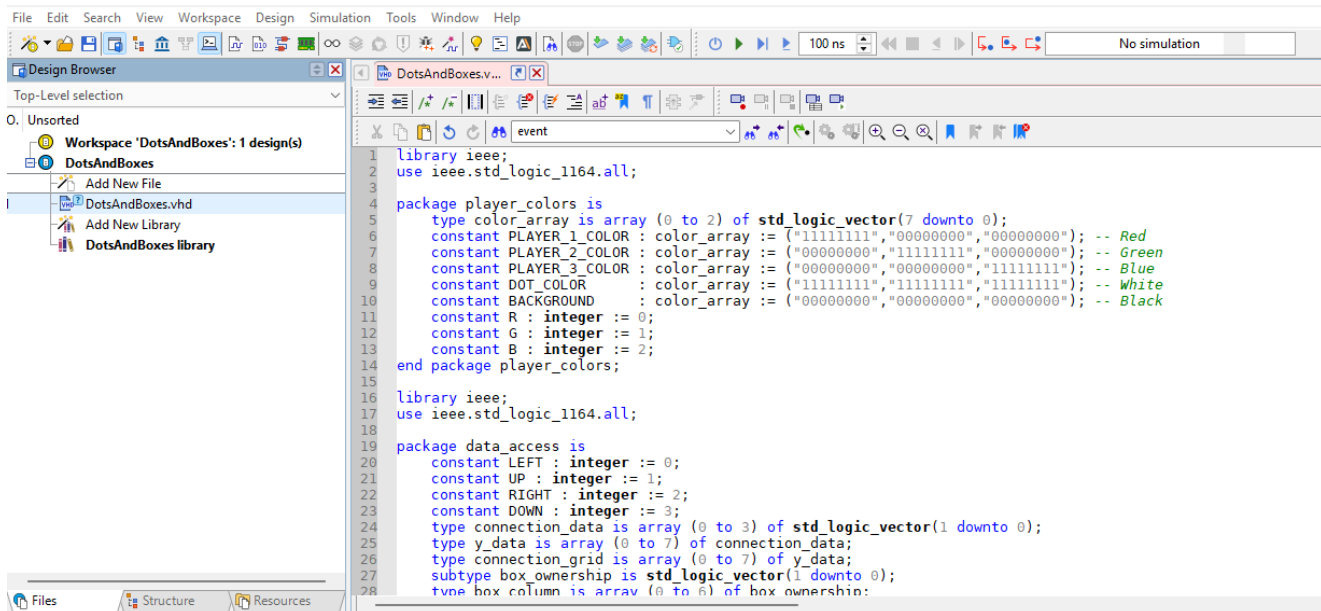
6. In the Design Browser, click on **Add New File**



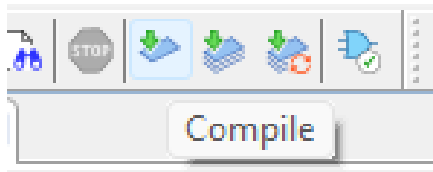
7. Click on **VHDL Source Code** and name the file **DotsAndBoxes**



8. Paste [the code](#) into the file



9. Save the file and compile the code. Ensure there are no errors



```
» # Compile Architecture "state_machine" of Entity "main_game_loop"
» # Compile success 0 Errors 0 Warnings Analysis time : 0.2 [s]
» # ELBREAD: Elaboration process.
» # ELBREAD: Elaboration time 0.0 [s].
»
```

Programming with Quartus

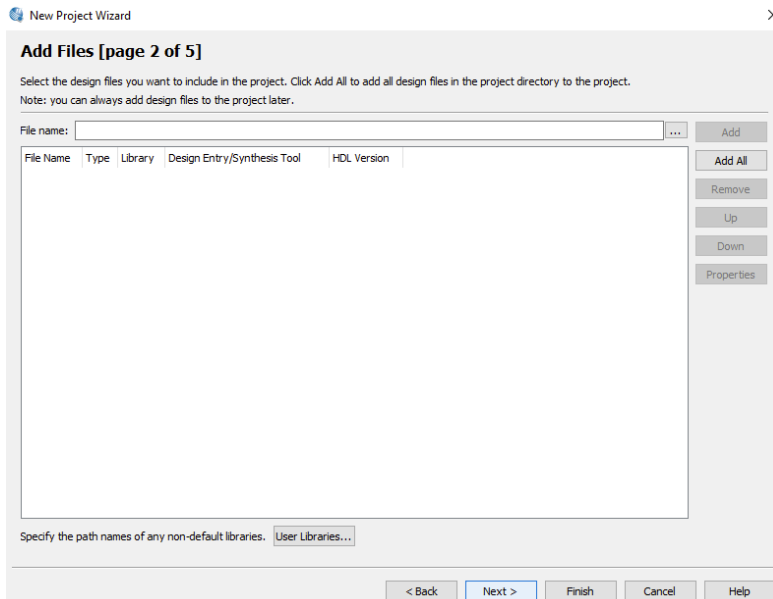
We will now use [Quartus II](#) (version 13.1 in this example) to once more compile our design, route the pin assignments, and program the FPGA.

Creating the project

1. In Quartus, go to **File > New Project Wizard...**
2. **Decide the working directory for the project, DotsAndBoxes, and name the project “main_game_loop”. The name must be this to be identical to the top-level module. Click Next.**

The screenshot shows the 'New Project Wizard' dialog box in Quartus II. The title bar reads 'New Project Wizard' with a close button. The main area is titled 'Directory, Name, Top-Level Entity [page 1 of 5]'. It contains three text input fields with prompts: 'What is the working directory for this project?' (containing 'C:\altera\13.1\DotsAndBoxes'), 'What is the name of this project?' (containing 'main_game_loop'), and 'What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.' (containing 'main_game_loop'). Each field has a browse button (three dots) to its right. Below these fields is a button labeled 'Use Existing Project Settings...'. At the bottom of the dialog are five buttons: '< Back', 'Next >', 'Finish', 'Cancel', and 'Help'. The 'Next >' button is highlighted with a blue border.

3. Click Next to skip the Add Files page as we will paste the code in later.



New Project Wizard

Add Files [page 2 of 5]

Select the design files you want to include in the project. Click Add All to add all design files in the project directory to the project.
Note: you can always add design files to the project later.

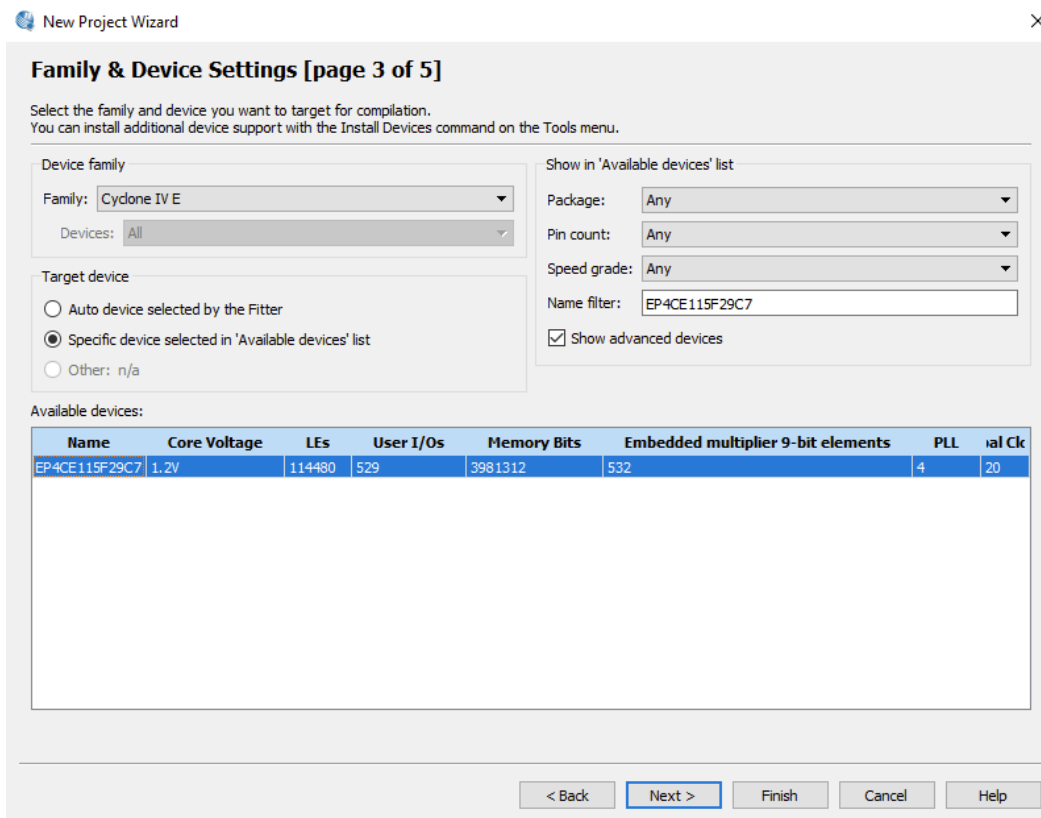
File name: ...

File Name	Type	Library	Design Entry/Synthesis Tool	HDL Version
-----------	------	---------	-----------------------------	-------------

Specify the path names of any non-default libraries.

< Back **Next >** Finish Cancel Help

4. Select the Cyclone IV E Device family on the Family & Device Settings page and type in the name filter: EP4CE115F29C7. Click on it in the Available devices list, and click Next.



New Project Wizard

Family & Device Settings [page 3 of 5]

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

Device family

Family:

Devices:

Target device

☐ Auto device selected by the Fitter

☒ Specific device selected in 'Available devices' list

☐ Other: n/a

Show in 'Available devices' list

Package:

Pin count:

Speed grade:

Name filter:

☒ Show advanced devices

Available devices:

Name	Core Voltage	LEs	User I/Os	Memory Bits	Embedded multiplier 9-bit elements	PLL	Max Clk
EP4CE115F29C7	1.2V	114480	529	3981312	532	4	20

< Back **Next >** Finish Cancel Help

5. Leave everything default on the EDA Tool Settings page, and click Next.

New Project Wizard

EDA Tool Settings [page 4 of 5]

Specify the other EDA tools used with the Quartus II software to develop your project.

EDA tools:

Tool Type	Tool Name	Format(s)	Run Tool Automatically
Design Entry/Synthesis	<None>	<None>	☐ Run this tool automatically to synthesize the current design
Simulation	ModelSim-Altera	VHDL	☐ Run gate-level simulation automatically after compilation
Formal Verification	<None>		
Board-Level	Timing	<None>	
	Symbol	<None>	
	Signal Integrity	<None>	
	Boundary Scan	<None>	

< Back Next > Finish Cancel Help

6. On the Summary page, it should look like this. Click Finish.

New Project Wizard

Summary [page 5 of 5]

When you click Finish, the project will be created with the following settings:

Project directory: C:\altera\13.1\DotsAndBoxes

Project name: main_game_loop

Top-level design entity: main_game_loop

Number of files added: 0

Number of user libraries added: 0

Device assignments:

Family name: Cyclone IV E

Device: EP4CE115F29C7

EDA tools:

Design entry/synthesis: <None> (<None>)

Simulation: ModelSim-Altera (VHDL)

Timing analysis: 0

Operating conditions:

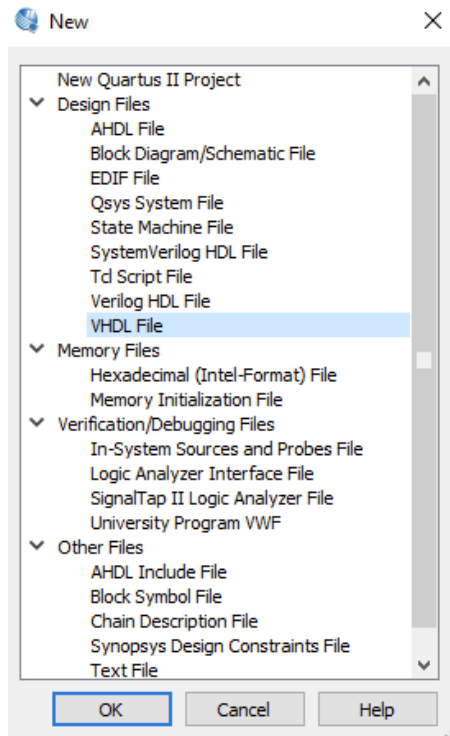
VCCINT voltage: 1.2V

Junction temperature range: 0-85 °C

< Back Next > Finish Cancel Help

Adding the source code

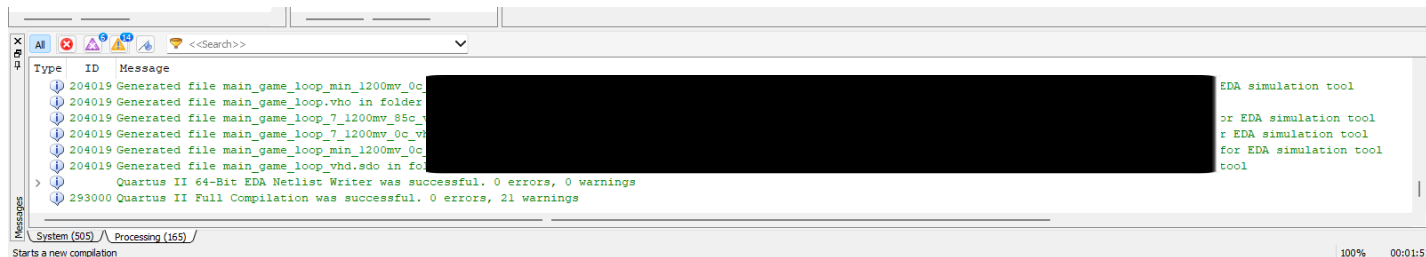
7. Go to File > New... and select VHDL File. Click OK.



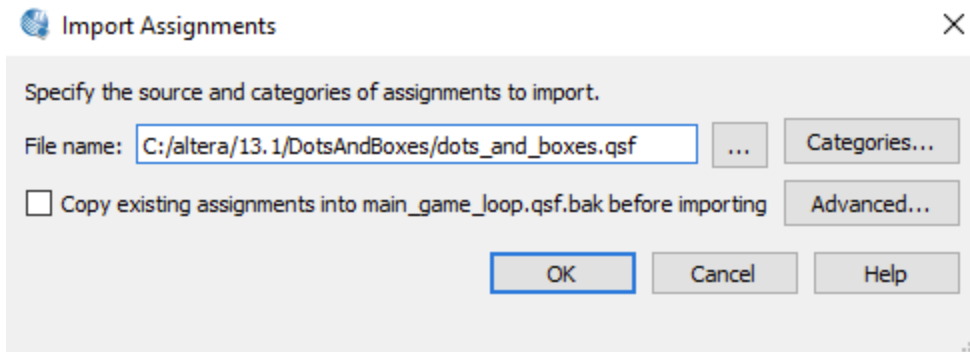
8. Paste in the code from the Active-HDL portion of this guide, or from the [Source Code section](#) of this document. Save it and name it anything you like.

Compilation and assignments

9. Go to Processing > Start Compilation and wait for successful compilation of the project with no errors (warnings are expected)



10. Copy the pin assignments from the [Pin Assignments section](#) of this document into a file, and name it "dots_and_boxes.qsf". Alternatively, skip to step 12 to do this part manually.
11. Go to Assignments > Import Assignments... and select the file from step 10. Uncheck the "Copy existing assignments" option. Click OK. Skip over the manual section of this guide to step 13.



12. Go to Assignments > Pin Planner and begin manually adding assignments until it looks as below.

	itatu:	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✓		 red_out[0]	Location	PIN_E12	Yes			
2	✓		 red_out[1]	Location	PIN_E11	Yes			
3	✓		 red_out[2]	Location	PIN_D10	Yes			
4	✓		 red_out[3]	Location	PIN_F12	Yes			
5	✓		 red_out[4]	Location	PIN_G10	Yes			
6	✓		 red_out[5]	Location	PIN_J12	Yes			
7	✓		 red_out[6]	Location	PIN_H8	Yes			
8	✓		 red_out[7]	Location	PIN_H10	Yes			
9	✓		 blue_out[7]	Location	PIN_D12	Yes			
10	✓		 blue_out[6]	Location	PIN_D11	Yes			
11	✓		 blue_out[5]	Location	PIN_C12	Yes			
12	✓		 blue_out[4]	Location	PIN_A11	Yes			
13	✓		 blue_out[3]	Location	PIN_B11	Yes			
14	✓		 blue_out[2]	Location	PIN_C11	Yes			
15	✓		 blue_out[1]	Location	PIN_A10	Yes			
16	✓		 blue_out[0]	Location	PIN_B10	Yes			
17	✓		 vs_out	Location	PIN_C13	Yes			
18	✓		 sync	Location	VREFGR..._B6_N0	Yes			
19	✓		 clk25_out	Location	PIN_A12	Yes			
20	✓		 clk_50	Location	PIN_Y2	Yes			
21	✓		 green_out[7]	Location	PIN_C9	Yes			
22	✓		 green_out[6]	Location	PIN_F10	Yes			
23	✓		 green_out[5]	Location	PIN_B8	Yes			
24	✓		 green_out[4]	Location	PIN_C8	Yes			

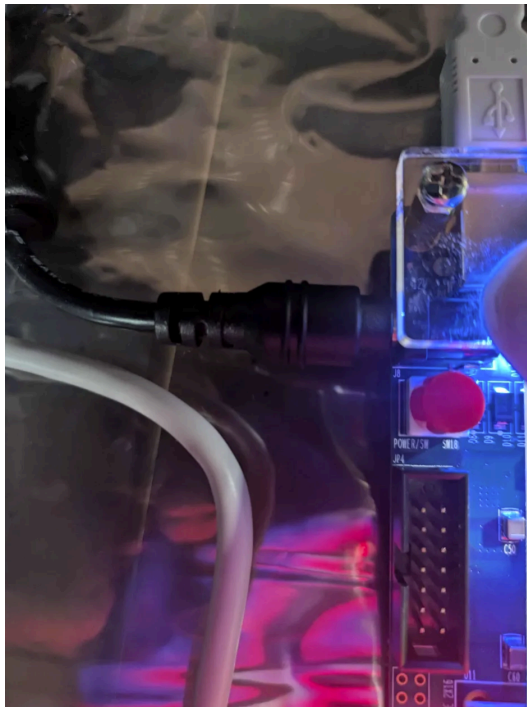
	itatu:	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
25	✓		 green_out[3]	Location	PIN_H12	Yes			
26	✓		 green_out[2]	Location	PIN_F8	Yes			
27	✓		 green_out[1]	Location	PIN_G11	Yes			
28	✓		 green_out[0]	Location	PIN_G8	Yes			
29	✓		 hs_out	Location	PIN_G13	Yes			
30	✓		 blank	Location	PIN_F11	Yes			
31	✓		 score_leds[0]	Location	PIN_E21	Yes			
32	✓		 score_leds[1]	Location	PIN_E22	Yes			
33	✓		 score_leds[2]	Location	PIN_E25	Yes			
34	✓		 score_leds[3]	Location	PIN_E24	Yes			
35	✓		 score_leds[4]	Location	PIN_H21	Yes			
36	✓		 score_leds[5]	Location	PIN_G20	Yes			
37	✓		 score_leds[6]	Location	PIN_G22	Yes			
38	✓		 score_leds[7]	Location	PIN_G21	Yes			
39	✓		 score_leds[8]	Location	PIN_F17	Yes			
40	✓		 player...eds[0]	Location	PIN_G19	Yes			
41	✓		 player...eds[2]	Location	PIN_E19	Yes			
42	✓		 player...eds[1]	Location	PIN_F19	Yes			
43	✓		 player...eds[3]	Location	PIN_F21	Yes			
44	✓		 player...eds[4]	Location	PIN_F18	Yes			
45	✓		 player...eds[5]	Location	PIN_E18	Yes			
46	✓		 player...eds[6]	Location	PIN_J19	Yes			
47	✓		 player...eds[7]	Location	PIN_H19	Yes			
48	✓		 player...eds[8]	Location	PIN_J17	Yes			

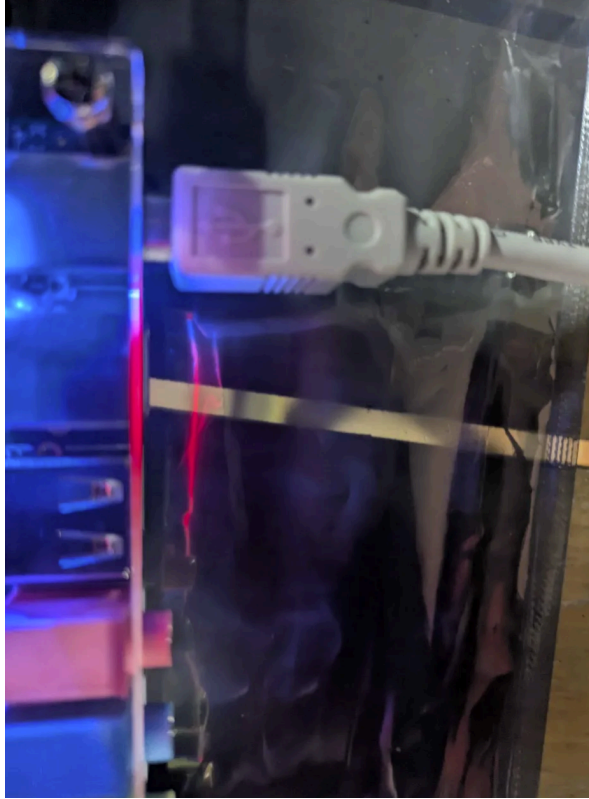
49	✓	out	player...eds[9]	Location	PIN_G17	Yes		
50	✓	out	player...ds[10]	Location	PIN_J15	Yes		
51	✓	out	player...ds[11]	Location	PIN_H16	Yes		
52	✓	out	player...ds[12]	Location	PIN_J16	Yes		
53	✓	out	player...ds[13]	Location	PIN_H17	Yes		
54	✓	out	player...ds[14]	Location	PIN_F15	Yes		
55	✓	out	player...ds[15]	Location	PIN_G15	Yes		
56	✓	out	player...ds[16]	Location	PIN_G16	Yes		
57	✓	out	player...ds[17]	Location	PIN_H15	Yes		
58	✓	in	left	Location	PIN_R24	Yes		
59	✓	in	up	Location	PIN_N21	Yes		
60	✓	in	down	Location	PIN_M23	Yes		
61	✓	in	right	Location	PIN_M21	Yes		
62	✓	in	y[0]	Location	PIN_AB28	Yes		
63	✓	in	y[1]	Location	PIN_AC28	Yes		
64	✓	in	y[2]	Location	PIN_AC27	Yes		
65	✓	in	x[0]	Location	PIN_AD27	Yes		
66	✓	in	x[1]	Location	PIN_AB27	Yes		
67	✓	in	x[2]	Location	PIN_AC26	Yes		
68	✓	in	reset	Location	PIN_Y23	Yes		

13. With the assignments added, go to Processing > Start Compilation again and wait for successful compilation.

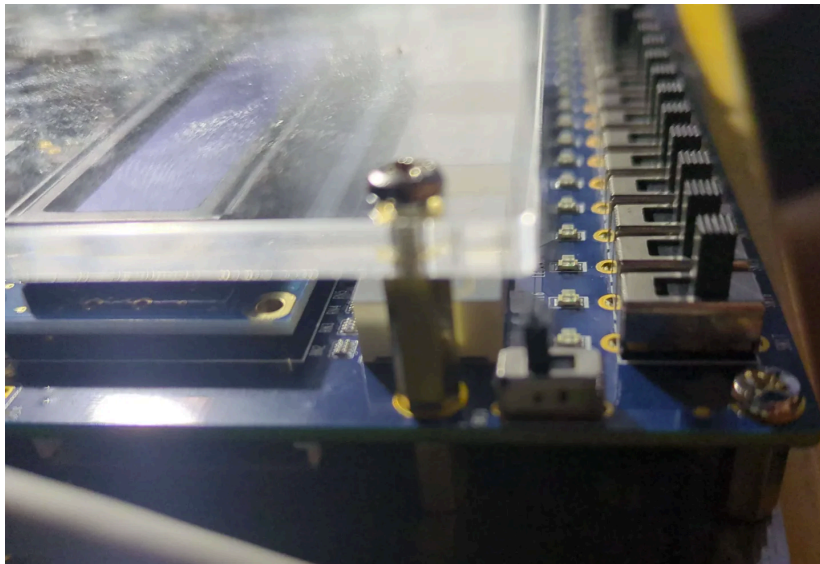
Programming

14. Power the DE2 board and plug it in via USB using the “BLASTER” port as pictured.



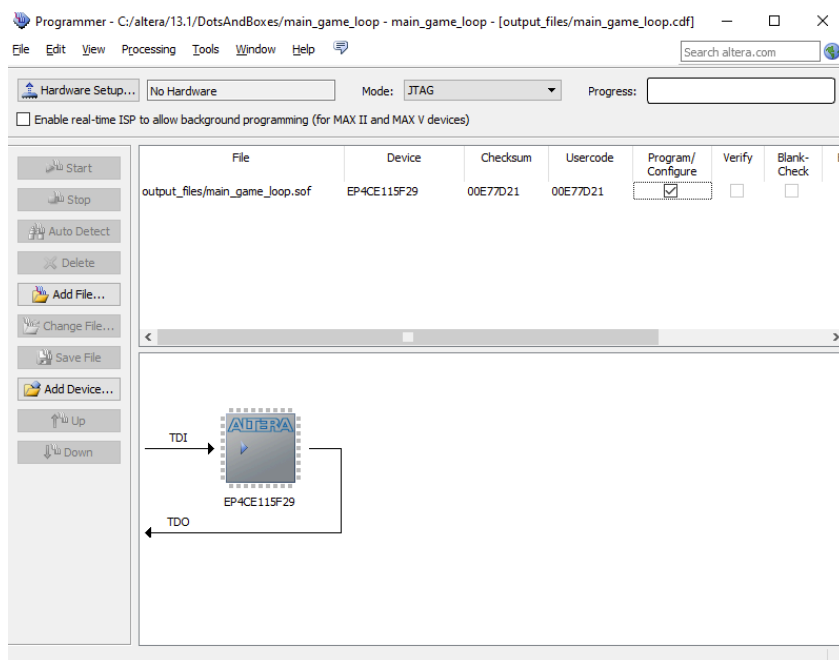


15. Ensure it is in RUN mode, not PROG mode. (Small switch below)

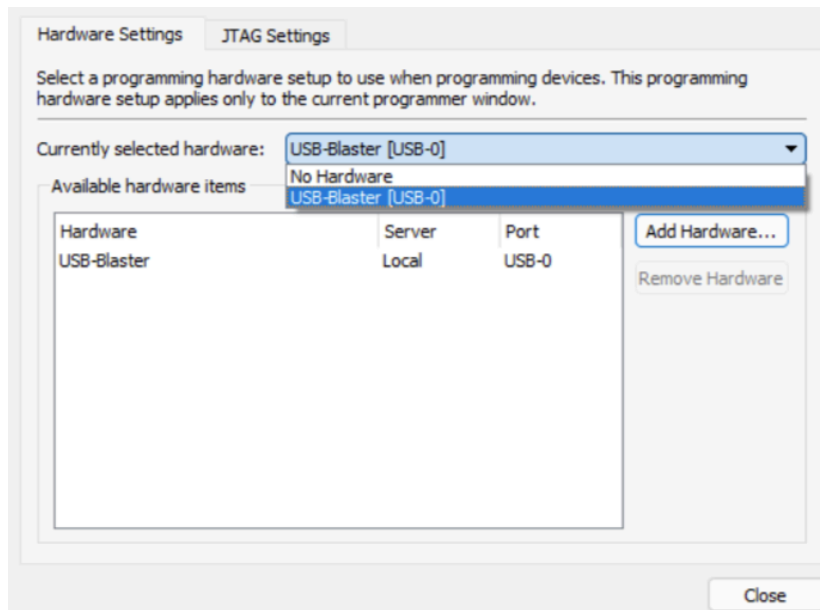


16. Power the monitor and connect it to the board via VGA cable.

17. Go to Tools > Programming and your screen should look something like this. The mode should be JTAG. If it says No Hardware, perform step 18.



18. Click on Hardware Setup... and change the Currently selected hardware from the drop down menu.



19. If there is no file shown, click Add File..., navigate to DotsAndBoxes\output_files and click on the .sof file.

20. Click start and wait for successful programming. The game is ready to be played!

Source Code

```
library ieee;
use ieee.std_logic_1164.all;

package player_colors is
    type color_array is array (0 to 2) of std_logic_vector(7 downto 0);
    constant PLAYER_1_COLOR : color_array :=
("11111111", "00000000", "00000000"); -- Red
    constant PLAYER_2_COLOR : color_array :=
("00000000", "11111111", "00000000"); -- Green
    constant PLAYER_3_COLOR : color_array :=
("00000000", "00000000", "11111111"); -- Blue
    constant DOT_COLOR      : color_array :=
("11111111", "11111111", "11111111"); -- White
    constant BACKGROUND     : color_array :=
("00000000", "00000000", "00000000"); -- Black
    constant R : integer := 0;
    constant G : integer := 1;
    constant B : integer := 2;
end package player_colors;

library ieee;
use ieee.std_logic_1164.all;

package data_access is
    constant LEFT : integer := 0;
    constant UP : integer := 1;
    constant RIGHT : integer := 2;
    constant DOWN : integer := 3;
    type connection_data is array (0 to 3) of std_logic_vector(1 downto 0);
    type y_data is array (0 to 7) of connection_data;
    type connection_grid is array (0 to 7) of y_data;
    subtype box_ownership is std_logic_vector(1 downto 0);
    type box_column is array (0 to 6) of box_ownership;
    type box_grid is array (0 to 6) of box_column;
end package data_access;

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.data_access.all;
```

```

entity main_game_loop is
    port(
        reset: in std_logic;
        clk_50: in std_logic;
        left: in std_logic;
        up: in std_logic;
        right: in std_logic;
        down: in std_logic;
        x: in std_logic_vector(2 downto 0);
        y: in std_logic_vector(2 downto 0);
        player_indicator_leds: out std_logic_vector(17 downto 0);
        score_leds: out std_logic_vector(8 downto 0);
        red_out: out std_logic_vector(7 downto 0);
        green_out: out std_logic_vector(7 downto 0);
        blue_out: out std_logic_vector(7 downto 0);
        hs_out: out std_logic;
        vs_out: out std_logic;
        sync: out std_logic;
        blank: out std_logic;
        clk25_out: out std_logic
    );
end main_game_loop;

architecture state_machine of main_game_loop is

    component display
        port(
            clk_50: in std_logic;
            red_out: out std_logic_vector(7 downto 0);
            green_out: out std_logic_vector(7 downto 0);
            blue_out: out std_logic_vector(7 downto 0);
            hs_out, vs_out, sync, blank, clk25_out: out std_logic;
            game_end: in std_logic;
            winning_player: in std_logic_vector(1 downto 0);
            connections: in connection_grid;
            boxes: in box_grid
        );
    end component;

    component led_player_indicator
        port(
            player: in std_logic_vector(1 downto 0);

```

```

        leds: out std_logic_vector(17 downto 0)
    );
end component;

component move_verification
    port(
        clk, reset: in std_logic;
        valid, invalid: out std_logic;
        x, y: in std_logic_vector(2 downto 0);
        direction: in std_logic_vector(3 downto 0);
        connections: out connection_grid;
        player: in std_logic_vector(1 downto 0)
    );
end component;

component scoring
    port(
        clk: in std_logic;
        reset: in std_logic;
        player: in std_logic_vector(1 downto 0);
        winner: out std_logic_vector(1 downto 0);
        score: out std_logic_vector(8 downto 0);
        x_cord, y_cord: in std_logic_vector(2 downto 0);
        direction: in std_logic_vector(3 downto 0);
        state: in std_logic_vector(1 downto 0);
        full, empty: out std_logic;
        connections: in connection_grid;
        boxes: out box_grid;
        player_scored: out std_logic
    );
end component;

signal connections: connection_grid := (others => (others => (others =>
(others => '0'))));
signal boxes: box_grid := (others => (others => (others => '0')));
signal player_latch, player, winning_player, current_player,
next_player: std_logic_vector(1 downto 0) := "01";
signal valid, invalid, valid_latch, invalid_latch, full, empty,
full_latch, empty_latch, game_end, button_pressed, player_scored, clk_25:
std_logic := '0';
signal direction, direction_latch : std_logic_vector(3 downto 0) :=
"0000";
signal current_state: std_logic_vector(1 downto 0) := "00";

```

```

signal x_latch, y_latch : std_logic_vector(2 downto 0) := "000";
signal state, next_state : std_logic_vector(1 downto 0) := "00";
begin
    clk25_out <= clk_25;
    game_loop : process (clk_50) is
    begin
        if (clk_50'event and clk_50 = '1') then
            full_latch <= full;
            empty_latch <= empty;
            valid_latch <= valid;
            invalid_latch <= invalid;
            if (reset = '1') then
                state <= "00";
                next_player <= "01";
            else
                state <= next_state;
            end if;
            case state is
                when "00" =>
                    if (button_pressed = '1') then
                        next_state <= "01";
                        player_latch <= current_player;
                        x_latch <= x;
                        y_latch <= y;
                        direction_latch <= direction;
                    else
                        next_state <= "00";
                    end if;
                    game_end <= '0';
                when "01" =>
                    if (valid_latch = '1') then
                        next_state <= "10";
                    elsif (invalid_latch = '1') then
                        next_state <= "00";
                    else
                        next_state <= "01";
                    end if;
                    game_end <= '0';
                when "10" =>
                    case current_player is
                        when "01" => next_player <= "10";
                        when "10" => next_player <= "11";
                        when "11" => next_player <= "01";
                    end case;
                end case;
            end case;
        end if;
    end process;
end game_loop;

```

```

        when others => next_player <= "01";
    end case;
    if (full_latch = '1') then
        next_state <= "11";
        next_player <= winning_player;
    elsif (empty_latch = '1') then
        next_state <= "00";
        if (player_scored = '1') then
            next_player <= current_player;
        end if;
    end if;
    game_end <= '0';
    when "11" =>
        next_state <= "11";
        game_end <= '1';
    when others =>
        next_state <= "00";
        game_end <= '0';
    end case;
    current_player <= next_player;
    current_state <= state;
    player <= current_player;
end if;
end process game_loop;

disp : display port map(clk_50, red_out, green_out, blue_out, hs_out,
vs_out, sync, blank, clk_25, game_end, winning_player, connections, boxes);
leds : led_player_indicator port map(player, player_indicator_leds);
verify : move_verification port map(clk_50, reset, valid, invalid,
x_latch, y_latch, direction_latch, connections, player_latch);
scorer : scoring port map(clk_50, reset, player_latch, winning_player,
score_leds, x_latch, y_latch, direction_latch, current_state, full, empty,
connections, boxes, player_scored);
direction <= "1000" when left = '0' else
    "0100" when up = '0' else
    "0010" when right = '0' else
    "0001" when down = '0' else
    "0000";
button_pressed <= not (left and up and down and right);
end state_machine;

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

```

```

use work.player_colors.all;
use work.data_access.all;
-- Manage VGA, displaying dot grid, colored connections, colored filled
boxes, and the winner at the end
entity display is
    port(
        clk_50: in std_logic;                -- DE2 Clock
        red_out: out std_logic_vector(7 downto 0);    -- VGA R
        green_out: out std_logic_vector(7 downto 0);  -- VGA G
        blue_out: out std_logic_vector(7 downto 0);   -- VGA B
        hs_out: out std_logic;                  -- Horizontal
Sync
        vs_out: out std_logic;                  -- Vertical
Sync
        sync: out std_logic;                   -- Unused
        blank: out std_logic;                  -- Tie to 1
for color
        clk25_out: out std_logic;              -- VGA Clock
        game_end: in std_logic;               -- Display
winner
        winning_player: in std_logic_vector(1 downto 0); -- Player:
R-01 G-10 B-11
        connections: in connection_grid;      -- Grid of
connections
        boxes: in box_grid                    -- Grid of
filled boxes
    );
end display;

architecture behavior of display is
    signal clk25 : std_logic := '0';          --
Internal Clock aligned with VGA
    signal horizontal_counter : integer := 0;  -- tracks what column we
are on
    signal vertical_counter : integer := 0;    -- tracks what row we are on
begin
    clk25_out <= clk25;
    sync <= '0';
    blank <= '1' when
        ((horizontal_counter < 640) and (vertical_counter < 480))
    else
        '0';

```



```

-- Divide the 50 MHz clock because VGA pixel clock is 25MHz
process (clk_50)
begin
    if clk_50'event and clk_50='1'
    then
        if (clk25 = '0')
        then
            clk25 <= '1';
        else
            clk25 <= '0';
        end if;
    end if;
end process;

-- Draw game state
process (clk25)
    variable dot_pixel : std_logic;           -- 1 where
grid should be drawn
    variable connection_pixel : std_logic;     -- 1
where a connection should be drawn
    variable connection_color : color_array;   -- color of
current connection
    variable connection_direction: integer;    -- left
up down right
    variable fill_pixel : std_logic;           -- 1 where
boxes are filled
    variable fill_color : color_array;         -- color
of current filled box
    variable final_color : color_array;        -- final
color of current pixel
    variable connection_num : std_logic_vector (1 downto 0);-- tracks
the player that owns connection
    variable box_num : std_logic_vector (1 downto 0);    -- tracks
box owner
begin
    if clk25'event and clk25 = '1'
    then
        dot_pixel := '0';
        connection_pixel := '0';
        fill_pixel := '0';

        ----- Check for Hsync and Vsync -----
        -- signals should be recieved in order: pixels, front porch, sync,
back porch

```

```

-- H Front porch: 0.6 us (15 clk cycles)
-- Horizontal timing spec: 3.8 us for Hsync (95 clk cycles) but
after 640 + 15 (front porch)
    if (horizontal_counter >= 655)
    and (horizontal_counter < 750) -- 655 + 95 clk cycles
    then
        hs_out <= '0';
    else
        hs_out <= '1';
    end if;
-- H Back porch: 1.9 us (47 clk cycles)

-- V Front porch: 10 lines
-- Vertical timing spec: 2 lines for Vsync but after 480 + 10
(front porch)
    if (vertical_counter >= 490)
    and (vertical_counter < 492) -- 490 + 2 (sync)
    then
        vs_out <= '0';
    else
        vs_out <= '1';
    end if;
-- V Back porch: 33 lines

----- Update pixel position -----
horizontal_counter <= horizontal_counter+1;
if (horizontal_counter=797) -- new line at 797, exactly 640 +
15 (front porch) + 95 (sync) + 47 (back porch)
then
    vertical_counter <= vertical_counter+1;
    horizontal_counter <= 0;
end if;
if (vertical_counter=525) -- reset at 525, exactly 480 + 10
(front porch) + 2 (sync) + 33 (back porch)
then
    vertical_counter <= 0;
end if;

----- Determine if we are on a dot -----
if (
    (((horizontal_counter - 20) mod 40) < 8) -- -20 pushes the
first dot forward

```

```

        and (horizontal_counter >= 160) -- 40 * 4, centering
        and (horizontal_counter < 488) -- 160 + 40 * 8 + 8
        and (((vertical_counter - 20) mod 40) < 8)
        and (vertical_counter >= 80) -- 40 * 2, centering
        and (vertical_counter < 408) -- 80 + 40 * 8 + 8
    )
then
    dot_pixel := '1';
end if;

        ----- Determine if we are on a connection
and set connection color -----
    -- check if within line draw region
    if (
        (horizontal_counter >= 182) -- 40 * 4 + 22, centering and
aligning
        and (horizontal_counter < 466) -- 182 + 40 * 7 + 4
        and (vertical_counter >= 102) -- 40 * 2 + 22, centering and
aligning
        and (vertical_counter < 386) -- 102 + 40 * 7 + 4
    )
then
    -- check if on a horizontal line
    if (((vertical_counter - 22) mod 40) < 4) -- -22 pushes the
line forward
    then
        -- check the connections of dot to left of us
        if
        (connections((horizontal_counter-182)/40)((vertical_counter-102)/40)(RIGHT)
        /= "00")
        then
            connection_pixel := '1';
            connection_num :=
connections((horizontal_counter-182)/40)((vertical_counter-102)/40)(RIGHT);
        end if;
        -- check if on a vertical line
        elsif (((horizontal_counter - 22) mod 40) < 4)
        then
            -- check the connections of dot above us
            if
            (connections((horizontal_counter-182)/40)((vertical_counter-102)/40)(DOWN)
            /= "00")
            then

```

```

        connection_pixel := '1';
        connection_num :=
connections((horizontal_counter-182)/40)((vertical_counter-102)/40)(DOWN);
        end if;
    end if;
end if;

-- Determine connection color
if (connection_pixel = '1')
then
    if (connection_num = "01")
    then
        connection_color := PLAYER_1_COLOR;
    elsif (connection_num = "10")
    then
        connection_color := PLAYER_2_COLOR;
    elsif (connection_num = "11")
    then
        connection_color := PLAYER_3_COLOR;
    else
        connection_color := BACKGROUND;
    end if;
end if;

----- Determine if we are on a filled box
and set fill color -----
    if (
        (
            (((horizontal_counter - 28) mod 40) < 32) -- -28 pushes the
first box forward
        and
            (((vertical_counter - 28) mod 40) < 32)
        )
        and (horizontal_counter >= 188) -- 40 * 4 + 28, centering
and aligning
        and (horizontal_counter < 460) -- 188 + 40 * 7 - 8
        and (vertical_counter >= 108) -- 40 * 2 + 28, centering and
aligning
        and (vertical_counter < 380) -- 108 + 40 * 7 - 8
    )
    then
        -- check the color of the box
        box_num :=

```

```

boxes((horizontal_counter-182)/40)((vertical_counter-102)/40);
    if (box_num = "01")
    then
        fill_color := PLAYER_1_COLOR;
        fill_pixel := '1';
    elsif (box_num = "10")
    then
        fill_color := PLAYER_2_COLOR;
        fill_pixel := '1';
    elsif (box_num = "11")
    then
        fill_color := PLAYER_3_COLOR;
        fill_pixel := '1';
    else
        fill_color := BACKGROUND;
    end if;
end if;

----- Final pixel color determination -----
if (game_end = '1') -- Display winner color
then
    if (winning_player = "01")
    then
        final_color := PLAYER_1_COLOR;
    elsif (winning_player = "10")
    then
        final_color := PLAYER_2_COLOR;
    elsif (winning_player = "11")
    then
        final_color := PLAYER_3_COLOR;
    else
        final_color := BACKGROUND;
    end if;
elseif (connection_pixel = '1') -- Display connections above
boxes
then
    final_color := connection_color;
elseif (fill_pixel = '1') -- Display filled boxes
then
    final_color := fill_color;
elseif (dot_pixel = '1') -- Display dot grid lowest
level
then

```

```

        final_color := DOT_COLOR;
    else
        final_color := BACKGROUND;
    end if;

    red_out <= final_color(R);
    green_out <= final_color(G);
    blue_out <= final_color(B);
end if;
end process;

end behavior;

----verification module--
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.data_access.all;
entity move_verification is
    port(
        clk, reset: in std_logic;    --clk needed to keep track of
game state
        valid, invalid: out std_logic;
        x, y: in std_logic_vector(2 downto 0);
        direction: in std_logic_vector(3 downto 0);
        connections: out connection_grid;
        player: in std_logic_vector(1 downto 0)
    );
end move_verification;

architecture verification of move_verification is
    signal connection_register: connection_grid := (others => (others =>
(others => (others => '0'))));

    --shared signals
    signal direction_index: integer range 0 to 3;
    signal opposite: integer range 0 to 3;
    signal x_next, y_next: integer range -1 to 8;    --extended both ways

```

```

for checking boundaries
    signal no_input: boolean;
    signal valid_internal: std_logic := '0';
begin
    connections <= connection_register;

    --process to set coordinates and direction
    process(x, y, direction, player)
        variable x_int, y_int: integer range 0 to 7;
    begin
        --convert coordinates to integers
        x_int := to_integer(unsigned(x));
        y_int := to_integer(unsigned(y));

        --default values
        no_input <= false;
        x_next <= x_int;
        y_next <= y_int;
        direction_index <= LEFT;
        opposite <= RIGHT;

        --decode direction
        if (direction(3) = '1') then           --LEFT
            direction_index <= LEFT;
            opposite <= RIGHT;
            x_next <= x_int - 1;
        elsif (direction(2) = '1') then       --UP
            direction_index <= UP;
            opposite <= DOWN;
            y_next <= y_int - 1;
        elsif (direction(1) = '1') then       --RIGHT
            direction_index <= RIGHT;
            opposite <= LEFT;
            x_next <= x_int + 1;
        elsif (direction(0) = '1') then       --DOWN
            direction_index <= DOWN;
            opposite <= UP;
            y_next <= y_int + 1;
        else
            --no direction
            --valid <= '0';
            --invalid <= '1';
            no_input <= true;
        end if;
    end begin;
end process;

```

```

        end if;
    end process;

    --process to check bounds and occupation status of line direction
    process(direction_index, opposite, x_next, y_next, no_input, x, y)
        variable x_int, y_int: integer range 0 to 7;
    begin
        x_int := to_integer(unsigned(x));
        y_int := to_integer(unsigned(y));

        --default values
        valid <= '0';
        invalid <= '0';
        valid_internal <= '0';

        if (no_input) then
            null;          -- do nothing

            --check bounds
            elsif (x_next < 0 or x_next > 7 or y_next < 0 or y_next > 7) then
                invalid <= '1';
            --check if occupied
            elsif (connection_register(x_int)(y_int)(direction_index) /= "00"
or connection_register(x_next)(y_next)(opposite) /= "00") then
                invalid <= '1';
            --move is valid
            else
                valid <= '1';
                valid_internal <= '1';
            end if;
        end process;

        --update logic
        --updates the grid if there's a valid move
        process(clk, reset)
            variable x_int, y_int: integer range 0 to 7;
        begin
            if (reset = '1') then
                connection_register <= (others => (others => (others => (others
=> '0'))));
            elsif (clk'event and clk = '1') then
                if (valid_internal = '1') then

```



```

        x_int := to_integer(unsigned(x));
        y_int := to_integer(unsigned(y));

        connection_register(x_int)(y_int)(direction_index) <=
player;
        connection_register(x_next)(y_next)(opposite) <= player;
    end if;
end if;
end process;
end verification;

```

```

library ieee;
use ieee.std_logic_1164.all;

```

```

entity led_player_indicator is
    port(
        player: in std_logic_vector(1 downto 0);
        leds: out std_logic_vector(17 downto 0)
    );
end led_player_indicator;

```

```

architecture player_indicator of led_player_indicator is
begin
    with player select
        leds <= (17 downto 12 => '1', others => '0') when "01",
        (11 downto 6 => '1', others => '0') when "10",
        (5 downto 0 => '1', others => '0') when "11",
        (others => '0') when others;
end player_indicator;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.data_access.all;

```

```

entity scoring is
    port(
        clk: in std_logic;
        reset: in std_logic;
        player: in std_logic_vector(1 downto 0);
        winner: out std_logic_vector(1 downto 0) := "00";
    );
end scoring;

```

```

        score: out std_logic_vector(8 downto 0);
        x_cord, y_cord: in std_logic_vector(2 downto 0);
        direction: in std_logic_vector(3 downto 0);
        state: in std_logic_vector(1 downto 0);
        full, empty: out std_logic := '0';
        connections: in connection_grid;
        boxes: out box_grid := (others => (others => (others => '0')));
        player_scored: out std_logic := '0'
    );
end scoring;

architecture score_keeper of scoring is
    signal P1_points, P1_points_internal: std_logic_vector(8 downto 0) :=
(others => '0');
    signal P2_points, P2_points_internal: std_logic_vector(8 downto 0) :=
(others => '0');
    signal P3_points, P3_points_internal: std_logic_vector(8 downto 0) :=
(others => '0');
    signal winning_player: std_logic_vector(1 downto 0) := "00";
    signal winning_score: std_logic_vector(8 downto 0) := (others => '0');
    signal current_score: std_logic_vector(8 downto 0) := (others => '0');
    signal filled: std_logic := '0';
    signal x, y : integer range 0 to 7 := 0;
    signal scored : std_logic := '0';
begin
    with reset select
        P1_points <= P1_points_internal when '0',
        (others => '0') when others;
    with reset select
        P2_points <= P2_points_internal when '0',
        (others => '0') when others;
    with reset select
        P3_points <= P3_points_internal when '0',
        (others => '0') when others;
    with winning_player select
        winning_score <= P3_points when "11",
        P2_points when "10",
        P1_points when "01",
        (others => '0') when others;
    with player select
        current_score <= P3_points when "11",
        P2_points when "10",
        P1_points when "01",

```

```

    (others => '0') when others;
winner <= winning_player;
score <= winning_score when (filled = '1') else current_score;
x <= to_integer(unsigned(x_cord));
y <= to_integer(unsigned(y_cord));
player_scored <= scored;
process(clk)
    variable points_to_add : integer := 0;
    variable new_score : std_logic_vector(8 downto 0) := "000000000";
    variable box1_x, box1_y, box2_x, box2_y, x_internal, y_internal :
integer range 0 to 7 := 0;
    variable direction_internal : integer range -1 to 3 := 0;
begin
    if (clk'event and clk = '1') then
        if (reset = '1') then
            boxes <= (others => (others => (others => '0')));
            P1_points_internal <= (others => '0');
            P2_points_internal <= (others => '0');
            P3_points_internal <= (others => '0');
            scored <= '0';
        end if;
        if (state = "10" and scored = '0') then
            box1_x := 7;
            box1_y := 7;
            box2_x := 7;
            box2_y := 7;
            x_internal := x;
            y_internal := y;
            points_to_add := 0;
            new_score := (others => '0');
            if (direction = "1000") then direction_internal := LEFT;
            elsif (direction = "0100") then direction_internal := UP;
            elsif (direction = "0010") then direction_internal := RIGHT;
            elsif (direction = "0001") then direction_internal := DOWN;
            else direction_internal := -1;
            end if;
            if (direction_internal = RIGHT) then
                x_internal := x + 1;
                direction_internal := LEFT;
            end if;
            if (direction_internal = LEFT) then
                if (x_internal /= 0 and y_internal /= 0) then
                    if (connections(x_internal)(y_internal)(UP) /= "00" and

```

```

connections(x_internal - 1)(y_internal - 1)(RIGHT) /= "00" and
connections(x_internal - 1)(y_internal - 1)(DOWN) /= "00") then
    box1_x := x_internal - 1;
    box1_y := y_internal - 1;
end if;
end if;
if (x_internal /= 0 and y_internal /= 7) then
    if (connections(x_internal)(y_internal)(DOWN) /= "00" and
connections(x_internal - 1)(y_internal + 1)(RIGHT) /= "00" and
connections(x_internal - 1)(y_internal + 1)(UP) /= "00") then
        box2_x := x_internal - 1;
        box2_y := y_internal;
    end if;
end if;
end if;
if (direction_internal = DOWN) then
    y_internal := y + 1;
    direction_internal := UP;
end if;
if (direction_internal = UP) then
    if (x_internal /= 0 and y_internal /= 0) then
        if (connections(x_internal)(y_internal)(LEFT) /= "00" and
connections(x_internal - 1)(y_internal - 1)(RIGHT) /= "00" and
connections(x_internal - 1)(y_internal - 1)(DOWN) /= "00") then
            box1_x := x_internal - 1;
            box1_y := y_internal - 1;
        end if;
    end if;
    if (x_internal /= 7 and y_internal /= 0) then
        if (connections(x_internal)(y_internal)(RIGHT) /= "00" and
connections(x_internal + 1)(y_internal - 1)(LEFT) /= "00" and
connections(x_internal + 1)(y_internal - 1)(DOWN) /= "00") then
            box2_x := x_internal;
            box2_y := y_internal - 1;
        end if;
    end if;
end if;
if (box1_x < 7 and box1_y < 7) then
    boxes(box1_x)(box1_y) <= player;
    points_to_add := points_to_add + 1;
end if;
if (box2_x < 7 and box2_y < 7) then
    boxes(box2_x)(box2_y) <= player;

```

```

        points_to_add := points_to_add + 1;
    end if;
    if (points_to_add /= 0) then
        scored <= '1';
    else
        scored <= '0';
    end if;
    new_score :=
std_logic_vector(to_unsigned(to_integer(unsigned(current_score)) +
points_to_add, new_score'length));
    case player is
        when "01" =>
            P1_points_internal <= new_score;
        when "10" =>
            P2_points_internal <= new_score;
        when "11" =>
            P3_points_internal <= new_score;
        when others =>
            null;
    end case;
    if (unsigned(P1_points_internal) + unsigned(P2_points_internal)
+ unsigned(P3_points_internal) = 49) then
        full <= '1';
        empty <= '0';
    else
        empty <= '1';
        full <= '0';
    end if;
else
    empty <= '0';
    full <= '0';
    scored <= '0';
end if;
end if;
if (P1_points > P2_points and P1_points > P3_points) then
    winning_player <= "01";
elsif (P2_points > P1_points and P2_points > P3_points) then
    winning_player <= "10";
elsif (P3_points > P2_points and P3_points > P1_points) then
    winning_player <= "11";
else
    winning_player <= "00";
end if;

```

```
end process;  
end score_keeper;
```

Pin Assignments

```
set_location_assignment PIN_E12 -to red_out[0]
set_location_assignment PIN_E11 -to red_out[1]
set_location_assignment PIN_D10 -to red_out[2]
set_location_assignment PIN_F12 -to red_out[3]
set_location_assignment PIN_G10 -to red_out[4]
set_location_assignment PIN_J12 -to red_out[5]
set_location_assignment PIN_H8 -to red_out[6]
set_location_assignment PIN_H10 -to red_out[7]
set_location_assignment PIN_D12 -to blue_out[7]
set_location_assignment PIN_D11 -to blue_out[6]
set_location_assignment PIN_C12 -to blue_out[5]
set_location_assignment PIN_A11 -to blue_out[4]
set_location_assignment PIN_B11 -to blue_out[3]
set_location_assignment PIN_C11 -to blue_out[2]
set_location_assignment PIN_A10 -to blue_out[1]
set_location_assignment PIN_B10 -to blue_out[0]
set_location_assignment PIN_C13 -to vs_out
set_location_assignment VREFGROUP_B6_N0 -to sync
set_location_assignment PIN_A12 -to clk25_out
set_location_assignment PIN_Y2 -to clk_50
set_location_assignment PIN_C9 -to green_out[7]
set_location_assignment PIN_F10 -to green_out[6]
set_location_assignment PIN_B8 -to green_out[5]
set_location_assignment PIN_C8 -to green_out[4]
set_location_assignment PIN_H12 -to green_out[3]
set_location_assignment PIN_F8 -to green_out[2]
set_location_assignment PIN_G11 -to green_out[1]
set_location_assignment PIN_G8 -to green_out[0]
set_location_assignment PIN_G13 -to hs_out
set_location_assignment PIN_F11 -to blank
set_location_assignment PIN_E21 -to score_leds[0]
set_location_assignment PIN_E22 -to score_leds[1]
set_location_assignment PIN_E25 -to score_leds[2]
set_location_assignment PIN_E24 -to score_leds[3]
set_location_assignment PIN_H21 -to score_leds[4]
set_location_assignment PIN_G20 -to score_leds[5]
set_location_assignment PIN_G22 -to score_leds[6]
set_location_assignment PIN_G21 -to score_leds[7]
set_location_assignment PIN_F17 -to score_leds[8]
set_location_assignment PIN_G19 -to player_indicator_leds[0]
set_location_assignment PIN_E19 -to player_indicator_leds[2]
set_location_assignment PIN_F19 -to player_indicator_leds[1]
```

```
set_location_assignment PIN_F21 -to player_indicator_leds[3]
set_location_assignment PIN_F18 -to player_indicator_leds[4]
set_location_assignment PIN_E18 -to player_indicator_leds[5]
set_location_assignment PIN_J19 -to player_indicator_leds[6]
set_location_assignment PIN_H19 -to player_indicator_leds[7]
set_location_assignment PIN_J17 -to player_indicator_leds[8]
set_location_assignment PIN_G17 -to player_indicator_leds[9]
set_location_assignment PIN_J15 -to player_indicator_leds[10]
set_location_assignment PIN_H16 -to player_indicator_leds[11]
set_location_assignment PIN_J16 -to player_indicator_leds[12]
set_location_assignment PIN_H17 -to player_indicator_leds[13]
set_location_assignment PIN_F15 -to player_indicator_leds[14]
set_location_assignment PIN_G15 -to player_indicator_leds[15]
set_location_assignment PIN_G16 -to player_indicator_leds[16]
set_location_assignment PIN_H15 -to player_indicator_leds[17]
set_location_assignment PIN_R24 -to left
set_location_assignment PIN_N21 -to up
set_location_assignment PIN_M23 -to down
set_location_assignment PIN_M21 -to right
set_location_assignment PIN_AB28 -to y[0]
set_location_assignment PIN_AC28 -to y[1]
set_location_assignment PIN_AC27 -to y[2]
set_location_assignment PIN_AD27 -to x[0]
set_location_assignment PIN_AB27 -to x[1]
set_location_assignment PIN_AC26 -to x[2]
set_location_assignment PIN_Y23 -to reset
```